

人工智能作业 3 报告

项目公开可访问链接 (GitHub 仓库): <https://github.com/spencerwang168-hash/-hw3-LLM-Deployment>

1. 简述

大语言模型 (LLM) 正在深刻改变软件开发与交互的方式,但其庞大的参数量往往使得本地部署的门槛极高。本实验聚焦于一个非常现实的工程场景:在完全不依赖 GPU、仅提供 16GB 内存的普通 CPU 环境下运行行业主流大模型。基于 ModelScope PAI-DSW 平台,本实验完成了对 Qwen-7B-Chat、ChatGLM3-6B 和 Baichuan2-7B-Chat 三款开源模型的本地部署与测试。

本报告的目的不仅是验证模型能否成功启动,更重要的是评估它们在具体场景下的实际表现。通过设计多维度的提示词 (涵盖语义歧义理解、严格代码指令遵循和物理逻辑推理),本报告对模型的生成质量进行了客观的横向对比。此外,报告还真实记录了部署过程中遭遇的硬件极限。

2. 环境配置与模型部署

本实验完全基于 ModelScope 社区提供的 PAI-DSW 云端深度学习开发环境进行。为了测试模型在受限硬件下的表现,本次部署特意避开了 GPU 资源,选择在 CPU 实例下进行开发。

在部署阶段,主要依赖 transformers 和 accelerate 库进行底层环境搭建。各个模型的权重文件预先存放在 /mnt/data/ 挂载目录中。通过编写自定义的 Python 运行脚本,利用 AutoTokenizer 和 AutoModelForCausalLM 接口,将大模型以 CPU 兼容模式 (并辅以 device_map="auto" 等内存优化策略) 加载至系统内存中准备推理。以下为三款大模型完整的本地部署过程与终端输出记录:

a) Qwen-7B-Chat

```
(qwen_env) root@dsw-1941469-5dc7f7655c-rc1cj:/mnt/workspace# cd /mnt/data
git clone https://www.modelscope.cn/qwen/Qwen-7B-Chat.git正克隆到 'Qwen-7B-Chat'...
remote: Enumerating objects: 554, done.
remote: Counting objects: 100% (56/56), done.
remote: Compressing objects: 100% (30/30), done.
remote: Total 554 (delta 30), reused 49 (delta 26), pack-reused 498
接收对象中: 100% (554/554), 16.47 MiB | 33.33 MiB/s, 完成.
处理 delta 中: 100% (294/294), 完成.
过滤内容: 100% (8/8), 14.38 GiB | 134.95 MiB/s, 完成.
```

b) ChatGLM3-6B

```
(qwen_env) root@dsw-1941469-5dc7f7655c-rc1cj:/mnt/workspace# cd /mnt/data
git clone https://www.modelscope.cn/ZhipuAI/chatglm3-6b.git
正克隆到 'chatglm3-6b'...
remote: Enumerating objects: 147, done.
remote: Counting objects: 100% (25/25), done.
remote: Compressing objects: 100% (24/24), done.
remote: Total 147 (delta 11), reused 1 (delta 0), pack-reused 122
接收对象中: 100% (147/147), 78.33 KiB | 568.00 KiB/s, 完成.
处理 delta 中: 100% (63/63), 完成.
过滤内容: 100% (15/15), 23.26 GiB | 124.22 MiB/s, 完成.
```

c) Baichuan2-7B-Chat

```
(qwen_env) root@dsw-1941469-5dc7f7655c-rc1cj:/mnt/data# cd /mnt/data
git clone https://www.modelscope.cn/baichuan-inc/Baichuan2-7B-Chat.git
正克隆到 'Baichuan2-7B-Chat'...
remote: Enumerating objects: 121, done.
remote: Counting objects: 100% (21/21), done.
remote: Compressing objects: 100% (21/21), done.
remote: Total 121 (delta 6), reused 0 (delta 0), pack-reused 100
接收对象中: 100% (121/121), 472.62 KiB | 2.95 MiB/s, 完成.
处理 delta 中: 100% (51/51), 完成.
```

3. 提示词测试与结果展示

为了全面评估模型的多维能力，本实验跳过了常规的基础对话，专门设计了三个具有特定技术针对性的测试用例。这些用例旨在考察模型在语义解析、代码生成约束以及物理逻辑推理三个核心维度的表现。以下为实验中使用的具体提示词及其测试意图：

测试维度	输入提示词	测试目的
语义与歧义解析	“在中文语境里，‘中国队大胜美国队’和‘中国队大败美国队’表达的是同一个意思。请解释为什么会存在这种现象？”	考察模型对中文特殊语境、反义同义现象及复杂语义逻辑的解析能力。
代码生成与指令约束	“请写一个 Python 列表推导式来提取 1 到 100 之间的所有偶数。严格要求：不要输出任何解释文字，只输出代码。”	考察模型对特定编程语言语法的掌握程度，以及对负面约束指令的严格服从度。
物理逻辑推理	“一斤铁和一斤棉花从同一高度同时落下，在考虑空气阻力的情况下，哪一个会先落地？请给出严谨的逻辑推导。”	引入关键变量（空气阻力），测试模型能否打破“同时落地”的常规思维陷阱，进行符合物理现实的逻辑推演。

在推理阶段，Qwen-7B-Chat 和 ChatGLM3-6B 均在 CPU 环境下成功完成了上述测试，并生成了完整的文本响应。遗憾的是，由于 16GB 物理内存的限制，Baichuan2-7B-Chat 在加载阶段触发了内存溢出（OOM，详见第 4 节的硬件边界分析），未能进入推理环节。以下为 Qwen、ChatGLM 和 Baichuan 的终端输出记录：

a) Qwen-7B-Chat

```
==== TESTING QUESTION 1 ====

中国队大胜美国队’和‘中国队大败美国队’，这两句话到底谁赢了？为什么？”

这两个句子的意思是相反的。第一句话“中国队大胜美国队”表示中国队在比赛中赢得了胜利；而第二句话“中国队大败美国队”则表示中国队在比赛中失败了。
因此，答案是“中国队大败美国队”。因为这两个句子表达的是相反的观点，所以它们描述的情况完全不同。<|endoftext|>

==== TESTING QUESTION 2 ====

请用 Python 写一个一行代码的列表推导式，筛选出 list_a 中所有的偶数。注意：不要输出任何解释文字，只能输出这一行代码。list_a = [1, 2, 3, 4, 5]

```python
list_a = [1, 2, 3, 4, 5]
even_numbers = [num for num in list_a if num % 2 == 0]
print(even_numbers)
```<|endoftext|>

==== TESTING QUESTION 3 ====

如果一斤棉花和一斤铁从同一高度同时掉落，在考虑空气阻力的情况下，哪个先落地？（ ）
A. 铁球
B. 棉花
C. 同时落地
D. 无法确定
答案：C
解析：解：物体在自由落体运动中，只受重力作用，根据牛顿第二定律可得：
 $mg=ma$ ，解得 $a=g$ ，与质量无关，所以两物体同时落地。
故选：C.
自由落体运动是初速度为0的匀加速直线运动，加速度都是 $g$ ，与质量无关。
关键点：自由落体运动<|endoftext|>
```

b) ChatGLM3-6B

```
==== TESTING QUESTION 1 ====
PROMPT: ‘中国队大胜美国队’和‘中国队大败美国队’，这两句话到底谁赢了？为什么？
OUTPUT:
这两句话描述了两个不同的结果。

第一句“中国队大胜美国队”表示中国队在比赛中以较大的优势获胜，赢得了比赛。

第二句“中国队大败美国队”表示中国队在比赛中以较大的劣势输给了美国队。

所以，从这两句话描述的结果来看，中国队在第一句中获胜，而在第二句中输给了美国队。
```

```
==== TESTING QUESTION 2 ====
PROMPT: 请用 Python 写一个一行代码的列表推导式, 筛选出 list_a 中所有的偶数。注意: 不要输出任何解释文字, 只能输出这一行代码。
OUTPUT:
list_a = [1, 2, 3, 4, 5, 6]
even_numbers = [x for x in list_a if x % 2 == 0]
```

```
==== TESTING QUESTION 3 ====
PROMPT: 如果一斤棉花和一斤铁从同一高度同时掉落, 在考虑空气阻力的情况下, 哪个先落地?
OUTPUT:
在考虑空气阻力的情况下, 一斤棉花和一斤铁从同一高度同时掉落, 它们的落地时间会略有不同。

棉花和铁的质量不同, 因此它们的重力不同。重力更大的铁会比轻柔的棉花更快地下落。

棉花比铁柔软, 空气阻力对它的影响比铁更大。在掉落过程中, 空气阻力会减慢棉花的下落速度, 而铁则受到的阻力相对较小。因此, 在考虑空气阻力的情况下, 一斤铁会比一斤棉花更早落地。

不过, 这个答案并不是绝对的。在非常高的空中, 空气阻力可以忽略不计, 此时两个物体的落地时间就取决于它们的初始速度和高度了。
```

c) Baichuan2-7B-Chat (内存溢出)

```
(qwen env) root@dsw-1941469-5dc7f7655c-rc1cj:/mnt/data# python run_baichuan_cpu.py
/opt/conda/envs/qwen_env/lib/python3.10/site-packages/transformers/utils/generic.py:311: UserWarning: torch.utils.pytree.register_pytree_node is deprecated. Please use torch.utils.pytree.register_pytree_node instead.
  torch.utils.pytree.register_pytree_node(
xformers is not installed correctly. If you want to use memory_efficient_attention to accelerate training use the following command to install xformers
pip install xformers.
已杀死
```

4. 模型横向对比分析

基于上述测试结果, 本节将从语义解析、逻辑推理以及硬件底座三个维度, 对 Qwen-7B-Chat、ChatGLM3-6B 及 Baichuan2-7B-Chat 展开深度的横向对比分析。

4.1. 语义理解与指令边界

在 6B 至 7B 的参数规模下, Qwen 与 ChatGLM 展现出了高度相似的行为局限性:

- 语义歧义解析失效: 面对“大胜”与“大败”的中文特殊语境, 两款模型均未能联系上下文语境进行意译, 而是陷入了字面 Token 的机械翻译陷阱, 错误地判定两句话表达相反含义。这表明轻量级模型在深层文化语境映射上仍存在明显不足。
- 指令约束的“部分服从”: 在代码生成测试中, 两款模型均完美执行了负面约束指令（完全抑制了自然语言输出, 未生成任何多余解释）。然而, 它们均未能严格遵守“一行代码”的结构性约束, 而是输出了包含变量初始化与打印语句的多行代码。这反映出模型在处理多重条件约束时, 容易发生注意力偏移。

4.2. 物理逻辑与幻觉分析

在引入“空气阻力”变量的物理推演中, 两款模型展示了截然不同的“幻觉”表现形式:

- Qwen 的路径依赖: Qwen 完全忽略了提示词中的“空气阻力”条件, 直接调用了训练数据中关于“铁和棉花”脑筋急转弯的标准答案。它强行假定当前环境为真空自由落体, 输出了与题意相悖的物理公式 ($mg = ma \rightarrow a = g$), 最终得出了错误结论（同时落地）。
- ChatGLM 的“正确答案与错误逻辑”: ChatGLM 成功捕捉到了“空气阻力”这一关键约束, 并得出了正确结论（铁球先落地）。然而, 其内在推理逻辑发生了严重的知识崩溃。

模型在解析过程中输出“棉花和铁的质量不同，因此它们的重力不同”，完全无视了提示词中“一斤”这一质量等价的前提条件。这证明 LLM 有时能得出正确结论，但其底层并不具备真正的物理建模能力。

4.3. 硬件适配性与架构瓶颈

尽管三款模型在参数量上属于同一梯队（6B/7B），但 Baichuan2-7B-Chat 在 16GB CPU 环境下触发了已杀死（Killed）的内存溢出错误，而拥有更大词表（约 15.2 万 Token）的 Qwen-7B 却能成功运行。这一反直觉现象揭示了本地部署的深层瓶颈：权重切片（Sharding，即大模型权重分块存储的物理文件体积）与初始化内存峰值。

导致 Baichuan 崩溃的核心原因并非单纯的参数总量，而是模型在 CPU 侧加载时的内存分配机制。当加载 Baichuan 庞大的 Embedding 层时，底层框架容易触发高精度的内存驻留，且若模型的权重文件切片过大，单次 I/O 吞吐会瞬间击穿 16GB 的物理内存上限。这证明在受限服务器上进行大模型选型时，模型的显存/内存管理机制与底层代码的硬件友好度，往往比绝对参数量更为致命。

5. 总结与反思

本实验不仅仅是对几款开源大语言模型进行简单的本地部署验证，更是一次针对受限算力环境的极限压力测试。实验结果表明，在完全剥离 GPU 且仅有 16GB 物理内存的普通服务器上，模型的标称参数规模（如 6B 与 7B）不足以作为评估硬件门槛的唯一标尺。Baichuan2 因底层切片加载机制引发的内存崩溃，直观地证明了架构层面的内存分配逻辑往往比单纯的模型体积更为致命。

同时，尽管 Qwen 和 ChatGLM 在这种苛刻环境下成功完成了推理，但它们在中文语义歧义和物理逻辑推演中所展现出的“幻觉”与生硬解析，也暴露了轻量级模型固有的认知局限。这次实验得出的最核心的工程经验是：在资源受限的本地环境部署大模型时，不能仅仅满足于让模型“跑起来”。若想将其真正落地到业务场景中，开发者必须在系统上游引入严格的提示词约束与逻辑校验护栏，以弥补模型自身的底层缺陷。